Flask Lezione 2

Introduzione

Nella lezione precedente abbiamo visto cos'è Flask, come definire le route statiche e dinamiche, e come utilizzare `url_for()` per generare URL dinamici.

In questa lezione approfondiamo l'architettura REST e come Flask la implementa.

Che cos'è REST

REST (REpresentational State Transfer):

- **è un'architettura per costruire applicazioni web che consente al client di richiedere risorse a un server.**
  Si basa sul modello Client-Server, dove il client invia richieste e il server gestisce dati e servizi.

Risorse

- **Una risorsa è qualsiasi elemento che può essere identificato e manipolato: pagine web, utenti, documenti, ecc.**
- Le risorse sono identificate da **[[Lezione 7 - Sistemi REST#**Struttura di un URI**|URI]]** (Uniform Resource Identifier).
  - **Un [[Lezione 7 - Sistemi REST#Struttura di un URI|URL]] è un tipo di URI che localizza anche la risorsa.**

> **Ogni URL è un URI, ma non tutti gli URI sono URL.**

In pratica, in applicazioni web RESTful, gli **endpoint** sono URI che il server espone per consentire l'accesso alle risorse.

Comunicazione HTTP

Le applicazioni REST utilizzano il protocollo **HTTP** per comunicare.
I principali **metodi HTTP** sono:

- `GET` → **recupera una risorsa**
- `POST` → **crea una nuova risorsa**
- `PUT` → **aggiorna una risorsa esistente**

- `DELETE` → **elimina una risorsa**

I dati scambiati tra client e server sono generalmente rappresentati in **JSON** o **XML**.

I principi fondamentali

Rest è uno standard per l'architettura Client-Server:

- Il client **è rappresentato, ad esempio, dal browser durante la navigazione web, tramite il quale inviamo richieste al server,**
- mentre il server **eroga servizi e gestisce le richieste e i dati.**
  REST si fonda su alcuni principi chiave:

1. **Stateless**:
   - il server non conserva informazioni sullo stato del client tra le richieste.
   - Ogni richiesta deve contenere tutte le informazioni necessarie per essere processata.
   - Questo riduce il carico sul server e semplifica la scalabilità.

> ✓ **Vantaggio: il server non si sovraccarica**

2. **Cacheable:**
   - **le risposte possono essere memorizzate temporaneamente dal client per ridurre il numero di richieste al server.**
   - Ad esempio, i browser utilizzano cache locali per migliorare le prestazioni.
3. **Uniform Interface**:
   - **l'interfaccia tra client e server è standardizzata, semplificando lo sviluppo e la manutenzione delle applicazioni**

Ogni richiesta del client deve essere **completa**, contenendo tutte le informazioni necessarie riguardo alla risorsa. La comunicazione avviene tramite il **protocollo HTTP**, utilizzando i metodi principali: `GET`, `POST`, `PUT` e `DELETE`.

> 📋 **URI vs. URL**
>
> - Qualsiasi elemento accessibile può essere considerato una risorsa: pagine web, utenti, documenti, ecc.
> - Le risorse sono identificate da un **URI (Uniform Resource Identifier)**.
> - Un **URL** è un tipo particolare di URI che localizza la risorsa su Internet.

- Tutti gli URL sono URI, ma non tutti gli URI sono URL.
- In pratica, molti URI corrispondono a endpoint, ma non tutti gli endpoint sono URI separati.
- Ad esempio, in Flask i path delle route sono URI, mentre durante la navigazione web utilizziamo URL per localizzarle.

REST: i vantaggi

REST offre diversi vantaggi grazie alla sua architettura standardizzata e al protocollo HTTP:

- **Semplicità**
  - **REST utilizza gli standard HTTP esistenti, semplificando lo sviluppo e la comprensione delle applicazioni web.**
- **Scalabilità**
  - **Essendo stateless, ogni richiesta è indipendente.**
  - **Il server non memorizza informazioni sullo stato del client, il che gli permette di gestire e servire più richieste contemporaneamente senza sovraccaricarsi.**
- **Flessibilità**
  - **REST supporta diversi formati per le richieste e le risposte.**
  - **Il formato più comune è il JSON, ma possono essere utilizzati anche altri formati come XML o YAML, a seconda delle esigenze dell'applicazione.**
- **Interoperabilità**
  - **REST funziona su qualsiasi piattaforma e con qualsiasi linguaggio di programmazione, a condizione che il client e il server utilizzino il protocollo HTTP.**
- **Separazione client-server**
  - **Client e server sono indipendenti: il client può evolvere senza modificare il server e viceversa, facilitando manutenzione, aggiornamenti e scalabilità dell'intero sistema.**

Risorse e URI

In REST, **qualsiasi elemento accessibile può essere considerato una risorsa**. Una risorsa rappresenta un'entità del dominio, quindi per analogia possiamo pensare a oggetti come quelli in analisi concettuale.

Ogni risorsa è identificata da un **[[Lezione 7 - Sistemi REST#**Struttura di un URI**|URI (Uniform Resource Identifier)]]**.
Gli URI dovrebbero essere **sostantivi**, non verbi, perché l'azione sulla risorsa è definita dai **verbi HTTP** (`GET` , `POST` , `PUT` , `DELETE`).
In altre parole:

- **gli URI rappresentano gli oggetti,**
- **mentre i** metodi HTTP **rappresentano le operazioni** .

> **Info**
>
> Se dovessimo accedere a una collezioni di Utenti che definisco un URI come `accediUtenti` è sbagliato, si deve solo scrivere `Utenti` .

Quando si progetta un URI:

- **Collezioni** → **usare il plurale** .
  - Ad esempio, per accedere a tutti gli utenti: `/utenti`
- **Singole risorse** → **aggiungere l'ID della risorsa.**
  - Ad esempio, per accedere all'utente con ID 5: `/utenti/5`
    Esempi:

HTTP

```
GET /libri        # Recupera tutti i libri (collezione)
GET /libri/5      # Recupera il libro con ID 5
POST /libri       # Crea un nuovo libro nella collezione
PUT /libri/5      # Aggiorna completamente il libro con ID 5
DELETE /libri/5   # Elimina il libro con ID 5
```

Esempi scorretti:

HTTP

```
GET /ottieniLibri     # URI con verbo → sbagliato
POST /creaLibro       # URI con verbo → sbagliato
DELETE /cancellaLibro/5  # URI con verbo → sbagliato
```

> **In sintesi:**
>
> - Gli **URI identificano le risorse**, non le azioni.

- Le **azioni** sono definite dai metodi HTTP.
- Le **collezioni** si scrivono al plurale, le **singole risorse** si identificano aggiungendo l'ID.

Questa convenzione aiuta a mantenere **coerenza, chiarezza e scalabilità** nell'architettura RESTful.

Metodi HTTP e operazioni CRUD(Create, Read, Update, Delete)

Le applicazioni REST operano sulle risorse utilizzando **operazioni CRUD** (Create, Read, Update, Delete).
Queste operazioni sono mappate sui **metodi HTTP** standard:

- **GET** → **leggere o recuperare risorse** (Read)
- **POST** → **creare nuove risorse** (Create)
- **PUT** → **aggiornare completamente una risorsa esistente** (Update)
- **PATCH** → **aggiornare parzialmente una risorsa** (Update)
- **DELETE** → **eliminare una risorsa** (Delete)

Ogni metodo ha comportamenti specifici riguardo **idempotenza** e **cache**, che ne determinano l'uso corretto all'interno di un'API RESTful.

Metodo HTTP	Operazione CRUD	Descrizione	Idempotente
GET	Read	Recupera Risorse	Si
POST	Create	Crea Nuove Risorse	No
PUT	Update	Aggiorna completamente una risorsa	Si
PATCH	Update	Modifica Parziale una risorsa	No
DELETE	Delete	Elimina la risorsa	Si

1. GET

- **Serve a recuperare risorse.**
- È **idempotente**: **eseguire più volte la stessa richiesta non cambia lo stato della risorsa.**

- È **cacheable**: **le risposte possono essere memorizzate localmente per ridurre le richieste al server.**

2. POST

- **Serve a creare e popolare nuove risorse all'interno di una collezione.**
- **Non è idempotente**: **inviare più volte la stessa richiesta genera più risorse.**
- Esempio: creare nuovi messaggi in una chat. Ogni POST genera un nuovo messaggio.

3. PUT

- **Serve a sostituire completamente una risorsa esistente.**
- È **idempotente**: **più richieste uguali producono lo stesso risultato.**
- Esempio: aggiornare tutti i campi di un utente o impostare il numero totale di pagine di un libro.

HTTP

```
PUT /libri/5
{
  "pagine": "110"
}
```

- **È utile quando si vuole aggiornare tutti i valori di una risorsa senza creare duplicati.**
- Esempio pratico: in WhatsApp, aggiornare una reaction a un messaggio con PUT permette di cambiare la reaction senza aggiungere una nuova emoji.

4. PATCH

- **Serve a modificare parzialmente una risorsa.**
- **Non è idempotente**: **più richieste consecutive possono produrre risultati diversi.**
- Esempio: aggiungere 10 pagine a un libro già esistente.

HTTP

```
PATCH /libri/5
{
    'pagine': "+10"
```

```
}
```

- In questo caso, ogni richiesta aggiunge 10 pagine in più, quindi il risultato cambia ad ogni esecuzione.
- **Con PATCH è possibile aggiornare solo campi specifici, o fare modifiche calcolabili, cosa che PUT non permette**.

Example

Esempio di PUT idempotente:

```http
PUT /libri/5 {   "pagine": "110" }
```

Eseguendo la stessa richiesta più volte, il risultato resta invariato.
Modificando il payload:

```http
PUT /libri/5 {   "pagine": "130" }
```

La risorsa viene completamente aggiornata.

5. DELETE

- **Serve a eliminare una risorsa.**
- È **idempotente**: **una volta cancellata la risorsa, ulteriori richieste DELETE non hanno effetto.**
- È necessario specificare quale risorsa eliminare, ad esempio `/libri/5`.

```http
DELETE /libri/5  # Elimina il libro con ID 5
DELETE /libri/5  # Nessuna azione: il libro non esiste più
```

Codici di Stato HTTP

Quando un **client** effettua una richiesta a un **server**, quest'ultimo risponde sempre con un **codice di stato HTTP**.

Il codice di stato indica l'esito della richiesta e permette al client di capire se l'operazione è andata a buon fine oppure se si è verificato un errore.

I codici di stato sono **standardizzati** e organizzati in famiglie, ognuna con un significato preciso.

La famiglia 200: 2xx - Successo

I codici della famiglia **2xx** indicano che la richiesta è stata elaborata correttamente dal server.

- **200 OK**
  - **La richiesta è stata completata con successo e il server restituisce una risposta.**
- **201 Created**
  - **Una nuova risorsa è stata creata correttamente.**
  - È tipicamente usato in risposta a una richiesta **POST**.
- **204 No Content**
  - **L'operazione è andata a buon fine, ma il server non restituisce alcun contenuto.**
  - È comune, ad esempio, dopo una richiesta **DELETE**.

La famiglia 400: 4xx - Errori Client

I codici 4xx indicano errori causati dal client, come richieste errate o mancanza di autorizzazioni.

- **400 Bad Request**
  - **La richiesta è malformata o non valida.**
  - **Ad esempio, il client invia un JSON non corretto (in Python può corrispondere a un `ValueError`).**
- **401 Unauthorized**
  - **Il client non è autenticato oppure non ha fornito credenziali valide.**
- **403 Forbidden**
  - **Il server ha compreso la richiesta, ma si rifiuta di eseguirla perché il client non ha i permessi necessari.**
- **404 Not Found**
  - **La risorsa richiesta non esiste oppure l'URI è errato.**
- **405 Method Not Allowed**

    - **Il metodo HTTP utilizzato non è consentito per quella risorsa.**
- **409 Conflict**
    - **Si è verificato un conflitto con lo stato attuale della risorsa, ad esempio in caso di aggiornamenti simultanei della stessa risorsa.**

5xx – Errori del Server

I codici **5xx** indicano errori che dipendono dal server.

- **500 Internal Server Error**
    - **Errore generico del server, causato da bug interni, errori di configurazione o problemi temporanei.**

I codici di stato HTTP fanno parte dello **standard del protocollo**.
È possibile definirne di personalizzati, ma è **sconsigliato**, perché si perderebbe interoperabilità e chiarezza nella comunicazione tra client e server.

JSON come Standard per REST

Il **JSON (JavaScript Object Notation)** **è lo standard più utilizzato per lo scambio di dati tra client e server nelle applicazioni REST.**
La sua struttura è semplice e immediatamente comprensibile, **poiché è simile a un dizionario Python o a un oggetto JavaScript.**
Questo rende il contenuto facilmente leggibile anche da un essere umano, permettendo di capire a colpo d'occhio quali dati vengono scambiati.

Uno dei principali vantaggi del JSON è il **supporto nativo in JavaScript**, che lo rende particolarmente adatto alle applicazioni web. Tuttavia, **il JSON è supportato praticamente da tutti i linguaggi di programmazione, che offrono strumenti per convertirlo automaticamente in oggetti o strutture dati interne**.

Il **parsing** del JSON è efficiente:

- **i dati vengono letti ed elaborati rapidamente dai calcolatori.**
  Inoltre, rispetto a formati come **XML**, il JSON è più **leggero e meno ridondante**, riducendo la quantità di dati trasmessi sulla rete.

Infine, il JSON ha una **struttura gerarchica**, che **permette di rappresentare in modo naturale dati complessi e annidati, come oggetti contenenti altri oggetti o liste.**

Esempio di rappresentazione JSON:

JSON

```
{
    'id': 1,
    "titolo": "1984",
    "autore": "George Orwell",
    "isbn": "978"
}
```

In questo esempio, un libro è rappresentato come una risorsa JSON, con coppie **chiave–valore** che descrivono le sue proprietà.

API REST: esempio pratico – Gestione di una libreria

Una API (Application Programming Interface):

- **è un insieme di regole e strumenti che permette a due software di comunicare tra loro.**
  In pratica, **un'API definisce quali richieste un client può fare a un server e come il server deve rispondere**.

Un'API è composta da un insieme di endpoint, ognuno dei quali è definito dalla combinazione di:

- un **[[Lezione 7 - Sistemi REST#**Struttura di un URI**|URI]]**
- un metodo HTTP

L'insieme dei verbi HTTP e degli URI costituisce l'API.

Base URL

Ogni API espone un URL**base**, che rappresenta il punto di accesso principale:

Plain text

```
https://api.biblioteca.it/v1
```

Tutti gli endpoint dell'API vengono definiti **a partire da questo URL base**.

Esempio di endpoint RESTful

HTTP

```
GET    /libri        # Restituisce la lista di tutti i libri
GET    /libri?genere=fantasy  # Restituisce i libri filtrati per genere
GET    /libri/123  # Restituisce il dettaglio del libro con ID 123
POST   /libri                  # Aggiunge un nuovo libro
PUT    /libri/123              # Aggiorna il libro con ID 123
DELETE /libri/123              # Elimina il libro con ID 123

GET    /autori                 # Restituisce la lista degli autori
GET    /autori/456/libri       # Restituisce i libri scritti dall'autore con ID 456
```

In questo esempio:

- `/libri` e `/autori` rappresentano **collezioni di risorse**
- `/libri/123` e `/autori/456` rappresentano **singole risorse**
- **Le azioni sulle risorse sono definite dai metodi HTTP, non dagli URI**

API RESTful

Quando un'API rispetta i principi REST (risorse, URI come sostantivi, uso corretto dei metodi HTTP, stateless), viene definita **[[Lezione 6 - API#•** Lezione 7 - Sistemi REST Sistemi REST REST (REpresentational State Transfer) **|API RESTful]]**.
Gli endpoint mostrati sopra sono RESTful perché:

- **utilizzano URI che rappresentano risorse**
- **usano i metodi HTTP per definire le operazioni**
- **seguono una struttura coerente e standardizzata**

Implementazione e documentazione

Quando si implementa un'API, per ogni endpoint devono essere definiti a livello di codice i relativi **metodi HTTP** (`GET` , `POST` , `PUT` , `DELETE`).
Queste informazioni vengono descritte in una **documentazione API**, spesso nel formato **OpenAPI**, che specifica:

- **gli endpoint disponibili**
- **i metodi supportati**
- **i parametri**
- **il formato delle richieste e delle risposte**

La documentazione permette ai client di sapere:

- **cosa è possibile fare**
- **e come interagire correttamente con il server.**

Richardson Maturity Model

Il **Richardson Maturity Model**:

- **è un modello che definisce 4 livelli di maturità (0–3) per valutare quanto un'API rispetti i principi REST.**
  In base alle caratteristiche dell'API, è possibile collocarla a uno specifico livello di maturità RESTful.

> **In generale, il cosiddetto "True REST" corrisponde al Livello 3, anche se la maggior parte delle API moderne si ferma al Livello 2.**

Livello 0 - The swamp of POX

Il **Livello 0**, noto come *The Swamp of POX* ("la palude del POX"), rappresenta il **livello più basso di maturità REST** secondo il Richardson Maturity Model.
POX significa *Plain Old XML* (o JSON):

- **indica un approccio molto semplice, ma poco strutturato, alla progettazione delle API.**
  Caratteristiche:

1. È presente **un solo endpoint**, ad esempio `/api`
2. Viene utilizzato **esclusivamente il metodo HTTP POST**
3. L'azione da eseguire **non è espressa dal verbo HTTP**, ma è specificata nel **payload** della richiesta
4. I verbi HTTP e i codici di stato **non vengono sfruttati in modo significativo**

In pratica, tutte le richieste vengono inviate allo stesso endpoint e il server è costretto a **analizzare il corpo della richiesta** per capire quale operazione eseguire.

Esempi:

JSON

```
POST /api
Content-Type: application/json
```

```
{
  "action": "getBook",
  "bookId": 5
}
```

JSON

```
POST /api
Content-Type: application/json
{
  "action": "createBook",
  "title": "Nuovo Libro",
  "author": "Autore"
}
```

In questo approccio:

- **tutte le operazioni passano dallo stesso endpoint**
- **la logica applicativa è contenuta nel body della richiesta**
- **il significato dell'operazione non è immediatamente deducibile dall'URL o dal metodo HTTP**

Problemi principali:

- ❌ **Non sfrutta i metodi HTTP (`GET`, `POST`, `PUT`, `DELETE`)**
- ❌ **Non utilizza correttamente i codici di stato HTTP**
- ❌ **È poco leggibile e difficile da mantenere**
- ❌ **Non consente un uso efficace della cache HTTP, con un impatto negativo sulle prestazion** i

Poiché ignora quasi completamente i principi fondamentali di REST e il funzionamento nativo del protocollo HTTP, il **Livello 0 non può essere considerato RESTful**.

Livello 1 : risorse multiple

Nel **Livello 1** del Richardson Maturity Model **vengono introdotte [[Lezione 7 - Sistemi REST#URI progressivi e risorse correlate|risorse multiple]], ciascuna identificata da un URI distinto.**
A differenza del Livello 0, l'API non espone più un unico endpoint generico, ma definisce **endpoint specifici per ogni risorsa**, ad esempio `/libri` o `/autori`.

Caratteristiche principali:

1. **Ogni risorsa è identificata da unURI diverso (es. `/libri`, `/autori`).**
2. **Le risorse diventano esplicite e riconoscibili tramite il percorso**
3. **Tuttavia, spesso viene ancora utilizzato un solo metodo HTTP, generalmente `POST`**

Il miglioramento rispetto al Livello 0 consiste quindi nella **separazione delle risorse**, che rende l'API più leggibile e meglio organizzata dal punto di vista strutturale.
Tuttavia, permangono alcune limitazioni importanti:

- ❌ **le operazioni non sono ancora chiaramente espresse dai verbi HTTP**
- ❌ **il server deve spesso analizzare il body della richiesta per capire quale azione eseguire (creazione, modifica, eliminazione).**

In pratica, **pur avendo URI diversi per ciascuna risorsa, la semantica delle operazioni non è ancora delegata al protocollo HTTP, ma rimane implicita nei dati della richiesta.**

Il Livello 1 rappresenta quindi un passo avanti rispetto al Livello 0, ma **non sfrutta ancora pienamente i principi REST**, poiché l'uso dei metodi HTTP è limitato e poco significativo.

Livello 2: HTTP + Codici di stato

Il Livello 2 del Richardson Maturity Model rappresenta il punto in cui un'API può essere considerata RESTful nella maggior parte dei casi pratici.

Caratteristiche

- utilizzo appropriato dei **verbi HTTP** (`GET`, `POST`, `PUT`, `DELETE`, ecc.)
- **uso degli URI come sostantivi, che rappresentano le risorse**
- **impiego corretto dei codici di stato HTTP per comunicare l'esito delle operazioni**

In questo livello **non si utilizza più esclusivamente il metodo POST**, ma si adottano i diversi metodi HTTP in base all'operazione da eseguire sulla risorsa. Ogni combinazione di **verbo HTTP + URI** definisce in modo chiaro e univoco l'azione richiesta.

Le API moderne rientrano in gran parte in questo livello, poiché garantisce un buon equilibrio tra **chiarezza, semplicità ed efficacia**.

Esempi:

```http
GET    /libri/5     → 200 OK
POST   /libri       → 201 Created
PUT    /libri/5     → 200 OK
DELETE /libri/5     → 204 No Content
GET    /libri/999   → 404 Not Found
```

A questo livello:

- **le azioni sono chiaramente espresse dai metodi HTTP**
- **l'esito delle operazioni è comunicato tramite codici di stato standard**
- **l'API risulta chiara, prevedibile e cacheabile**

👉 **La maggior parte delle API REST utilizzate in produzione opera al Livello 2**, poiché soddisfa i principi REST fondamentali senza introdurre complessità eccessiva.

Livello 3: HATEOAS

Il Livello 3 del Richardson Maturity Model introduce il concetto di **HATEOAS** (*Hypermedia As The Engine Of Application State*), che rappresenta il livello più alto di maturità REST.

Concetto chiave

A questo livello, è il server a guidare il client attraverso l'applicazione, fornendo nella risposta link ipermediali che descrivono le azioni possibili sullo stato corrente della risorsa.

Il client:

- **non deve conoscere a priori tutti gli endpoint**
- può **scoprirli dinamicamente** analizzando i link inclusi nella risposta del server

In altre parole, **il client non costruisce manualmente gli URI delle richieste successive, ma segue i link forniti dal server, in modo simile a come un**

browser web segue i collegamenti tra le pagine.

Funzionamento

Il Livello 3 include tutte le caratteristiche del **Livello 2** (URI come risorse, verbi HTTP corretti, codici di stato), ma aggiunge un ulteriore elemento:

- **la risposta del server contiene informazioni sulle operazioni disponibili**
- **tali operazioni sono rappresentate come link ipermediali**

Il server, quindi, non si limita a restituire i dati, ma **descrive anche come il client può interagire ulteriormente con la risorsa**.

Esempio di risposta HATEOAS

JSON

```
HTTP/1.1 201 Created
Content-Type: application/json
{
  "id": 5,
  "titolo": "1984",
  "_links": {
    "self":   { "href": "/libri/5" },
    "edit":   { "href": "/libri/5", "method": "PUT" },
    "delete": { "href": "/libri/5", "method": "DELETE" },
    "author": { "href": "/autori/42" },
    "reviews": { "href": "/libri/5/recensioni" }
  }
}

```

In questo esempio, oltre ai dati della risorsa, il server indica:

- dove recuperare la risorsa stessa (`self`).
- come modificarla (`edit`).
- come eliminarla (`delete`).
- come navigare verso risorse correlate (`author` , `reviews`).

✓ **Vantaggi**

- client **dinamico e autodescrittivo**
- possibilità di **evolvere l'API senza introdurre breaking changes**

- **navigabilità simile al web**, basata su link

✕ **Svantaggi**

- maggiore **complessità di progettazione e implementazione**
- **payload più grandi**
- **adozione limitata** nella pratica

Per questi motivi, sebbene il Livello 3 rappresenti il cosiddetto **“True REST”**, la maggior parte delle API REST moderne **si ferma al Livello 2**, considerato un compromesso efficace tra semplicità e aderenza ai principi REST.